01 · AI FOUNDATIONS · PROJECT REPORT

1. Title + links

Sunday League Fair Draw

Author: Tarish Abubakar

GitHub repository: https://github.com/khern123/Sunday-league-fair-draw

App: Local app, run instructions below.

A code-owned, reproducible way to turn a confirmed attendance list into balanced teams, with optional AI assistance for reading attendance chat and writing announcements.

02

2. Motivation

This app is for my Sunday mini league, where the organizer needs a quick and trusted way to turn a changing attendance list into teams.

Who has the problem: players and the organizer in the Sunday mini league. Attendance arrives informally, often through a group chat, and the final list can change close to kickoff.

Current process: Captains pick in turns. This description comes from the supplied demo feedback form.

When the process is manual, it can consume time during the narrow window before a match. It can also create arguments about who was included, whether teams are balanced, and whether the same people are repeatedly grouped together. The larger cost is trust: players cannot easily reproduce or verify the decision after it is made.

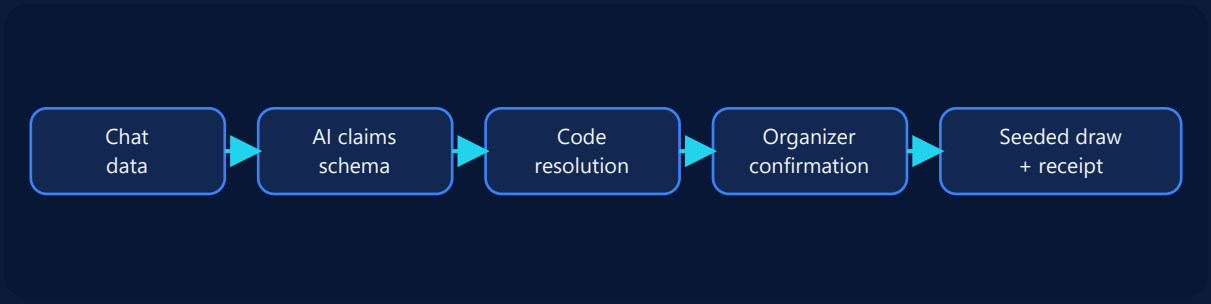
Local evidence: The supplied demo recorded 4 players, 26 minutes with the app, 2 redraws, and team balance as the main frustration. The 2-minute previous-process figure was explicitly marked as an estimate, not timed.

The project addresses that trust problem by requiring organizer confirmation, calculating teams in code, publishing a seed and receipt, and keeping optional AI assistance separate from the draw decision.

03

3. What is new here

The design separates interpretation from decisions: code owns parsing, attendance resolution, team creation, and verification; AI is optional and constrained to defined language tasks.



Comparison

[MY INPUT NEEDED: insert the verified list of existing products you compared against]

For each product, record only what was checked: what it does, whether it supports this league's attendance workflow, whether its team assignment is reproducible from a published seed, and whether it provides a public verification receipt.

[MY INPUT NEEDED: competitor comparison table and source links]

What this app does differently

The implemented design makes the boundary explicit: code parses timestamps and senders, resolves claims, requires organizer confirmation, creates the draw, and verifies the result. The optional model reads attendance chat and writes an announcement; it does not choose teams or assign ratings.

The draw is seeded and the public receipt can be checked later. I did not find one that does X until the comparison list and checks are supplied:

[MY INPUT NEEDED: replace “X” with a verified comparison finding, or state that no such checked difference was found]

04

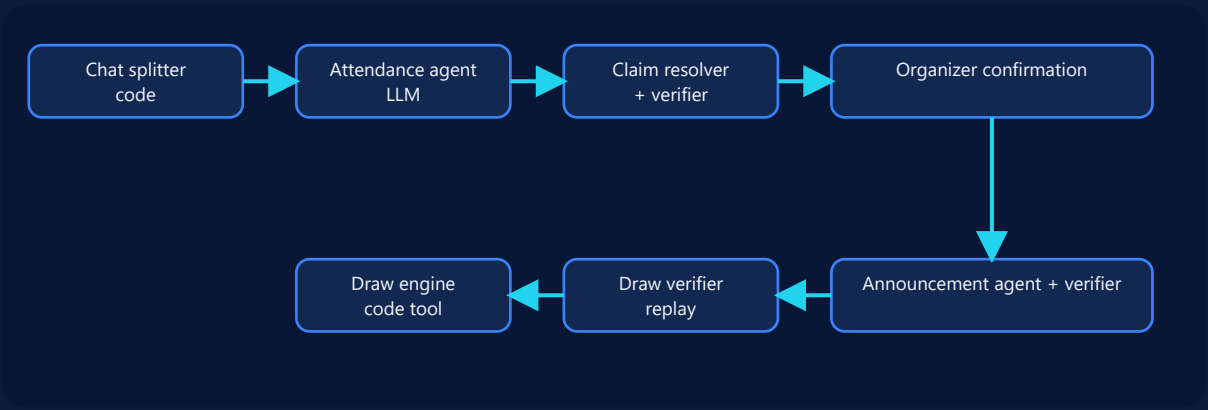
4. The agents

The system uses small, bounded components. Each model output is treated as untrusted data and is validated before it can affect the application.

Components and allowed tools

- **Chat splitter — code:** parses supported timestamps and senders, attaches continuation lines to the previous message, applies limits, and reports skipped or truncated lines. It uses no model tools.
- **Attendance agent — model:** receives numbered message indexes and text plus roster names and aliases; returns only schema-validated attendance claims. It uses only the centralized model client.
- **Claim resolver — code:** resolves exact names and aliases, applies only the latest self claim, flags third-party and mass claims, and creates pending guests without ratings. It uses no model tools.
- **Claim verifier — code:** checks message indexes, verbatim evidence, allowed statuses, and forbidden extra fields. It uses no model tools.
- **Organizer confirmation — human control:** reviews and edits proposed attendance, confirms the squad, and types guest ratings. It is not an autonomous agent.
- **Draw engine — code tool:** `fairdraw.draw.draw()` creates teams from the confirmed roster, settings, history, and seed. It uses no tokens and no model tools.
- **Draw verifier — code:** checks attendance coverage, unknown names, team-size balance, and seed replay. It uses no model tools.
- **Announcement agent — model:** receives only the public receipt and sanitized organizer note, then writes a short announcement. The code appends the exact seed, draw number, and hash.
- **Announcement verifier — code:** checks names and numbers against the allowed receipt data; after one retry it shows a code-generated fallback if needed.
- **Optional assistant agent — organizer-only model:** may plan only these allowlisted calls: `propose_attendance(chat_text)`, `set_status(player, status)`, `add_guest(name)`, `confirm_attendance()`, `run_draw(num_teams, seed_mode)`, `verify_draw()`, `write_announcement(note)`, and `save_night()`. Code validates and executes calls one at a time, with approval for saved-state changes.

Organization and handoffs



Model choice

The course key uses **gpt-6-luna** through the centralized Portkey client. A small model is sufficient for the language tasks because the draw, validation, seed replay, ratings, and team assignment remain code-owned.

One real run

[MISSING EVIDENCE: trace panel screenshot and screenshots showing each agent's real output]

Walkthrough to complete from the trace: chat splitter → attendance agent → claim resolver → claim verifier → organizer confirmation → draw engine → draw verifier → announcement agent → announcement verifier. No real run is described here until the trace and screenshots are attached.

05

5. Tokenomics

The recorded token measurement did not complete: `results/token_runs.json` has status **refused** because the planned 20 calls exceeded the remaining session budget. Therefore this report does not claim a typical-run token count, cost per run, or monthly user cost.

Prices are not set. The scaling report states: “prices not set; fill config.PRICING from course materials.” No cost number is invented here.

The intended cost controls are batching attendance messages, the application cache, centralized per-run/session/daily call budgets, and keeping the draw engine in code. The draw engine uses zero tokens. The cost experiment was also refused before calls because variant A required 30 calls in one run, above the configured per-run limit of 8.

Did cost cutting make the app worse? No measured comparison can be claimed because `results/cost_experiment.json` contains a refusal rather than accuracy results. This is a limitation, not evidence that batching or caching improves accuracy.

[MISSING EVIDENCE: completed token_runs table, cost experiment table, and token-panel screenshot]

06

6. Safety

What could go wrong

Chat can contain prompt-injection text or attempts to game attendance. A model can invent a player, quote, rating, or action. An organizer command could request an unsupported roster edit or create a loop. Repeated clicks or malformed input could overspend. Private ratings and real player data could also leak into public outputs.

Controls built into the system

Messages are separated from instructions and sanitized. The schema has no team or rating fields. Code parses timestamps and senders, resolves exact names and aliases, rejects mass claims and unknown names, keeps third-party claims pending confirmation, and gives guests no rating until the organizer enters one. The draw is code-only and cannot run before confirmation. Claims, draws, and announcements have separate verifiers. The model gateway has timeout/retry limits, application caching, and per-run, session, and daily budgets. Public receipts omit private ratings.

Evidence that defenses work

The attack, malfunction, and overspending suites reported **7 passed** in the recorded targeted run. The evaluator's safety metrics were: third-party or mass claims wrongly auto-applied **0**; injection attempts changing attendance or ratings **0**; unknown names wrongly accepted **0**. Source: targeted test output and `results/eval_summary.json`.

Test/risk	What was blocked
Instruction injection	Untrusted text was logged as a defense event and did not create a claim or action.
Fake system message	Role-spoofing text remained message data.
Third-party claim	Held for organizer confirmation; not auto-applied.
Mass claim	Rejected and logged.
Guest rating in chat	Guest remained pending without a rating.
Unknown player	Flagged and not added to the roster.

Overspending	Budget preflight refused experiments exceeding configured limits.

[MISSING EVIDENCE: `verified-badge-1440.png`, `mismatch-badge-1440.png`, and completed defense-demo screenshots]

07

7. Impact

Simulation

Source: `results/simulation_summary.json` and `results/simulation_table.md`. The sample roster was used; settings were 20 nights per repeat, 50 repeats, 0.8 attendance probability, minimum 10 attendees, and 300 candidates per draw.

Method	Strength gap	Top two same team	Max pair repeats	Avg top-3 teammates	Repeat score
A random	1.132 ± 0.177	56.8% ± 12.4%	12.90 ± 1.53	12.97 ± 0.14	1.053 ± 0.065
B balance	0.089 ± 0.019	56.2% ± 9.9%	11.72 ± 1.09	12.63 ± 0.15	1.063 ± 0.061
C balance + memory	0.096 ± 0.021	55.8% ± 9.9%	10.58 ± 0.88	12.67 ± 0.00	0.972 ± 0.061

C reduced repeated pairings compared with A and B, while B produced the smallest strength gap. These are simulated results, not a league-night experiment.

Attendance evaluation

Source: `results/eval_summary.json`, five runs on the labeled chat. Claim precision was **0.7500**, recall **0.1364**, F1 **0.2308**, and final attendance accuracy **0.2143**. The low recall shows that the evaluated model omitted many expected claims.

Real league night

The supplied demo recorded 4 players, 26 minutes with the app, 2 redraws, fairness ratings of 1/5 for each respondent, and “more” trust responses from each respondent. The earlier 2-minute figure was an estimate, not a timed measurement. No quotes were supplied.

08

8. Limits + next steps

The labeled evaluation contains real errors, not just abstract risks. Source: `results/eval_errors.md`. The recorded run produced 100 error entries across five runs; examples include expected “self/in”, “self/out”, “self/maybe”, “self/late”, guest, and third-party claims being returned as empty by the model.

Ratings are subjective inputs. Outsiders cannot independently replay the draw without the organizer’s private ratings and full receipt inputs. An organizer could still bias the result by editing attendance, ratings, or the roster before confirmation. One league night would be a small sample, and the simulation uses a small sample roster with simulated attendance.

The project also lacks completed real token and cost experiments because the safety budgets correctly refused them. Screenshot evidence for verification states, token usage, fallback behavior, and some draw states is still incomplete.

Next month: complete a permissioned real-night study; improve the attendance prompt and evaluate it against reviewed labels; complete a budget-approved cost comparison; add independent organizer/auditor review; improve the evidence capture flow; and test the public receipt verification process with an outside reviewer.